

Name: Monisha Raghini Chelle

RollNo:2520030537

Section -6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

6.To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
linux-System-Programming-Week1  OSSP-Practical      foldername  moni      sample1.c
SLAB                             Simple-Commands-Week1 hello       monis     sample1.c.save
SLAB14                           example.txt         hello.c     sample1
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
SLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
ipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
```

Step2 – Create the C file

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano escape_parser.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 escape_parser.c *
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

struct Token
{
    char value[MAX_TOKEN_LENGTH];
};

void tokenize(char *input, struct Token tokens[], int *count)
{
    int i = 0;
    int j;

    *count = 0;

    while (input[i] != '\0')
    {
        /* Skip whitespace */
        while (isspace((unsigned char)input[i]))
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        j = 0;

        /*
         * Read a token.
         * Backslash protects the next character.
         */
        while (input[i] != '\0' &&
               !isspace((unsigned char)input[i]))
        {
            if (input[i] == '\\' && input[i + 1] != '\0')
            {
                i++;
            }
        }
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 escape_parser.c *

        i++;

        if (j < MAX_TOKEN_LENGTH - 1)
        {
            tokens[*count].value[j++] = input[i];
        }

        i++;
    }
else
{
    if (j < MAX_TOKEN_LENGTH - 1)
    {
        tokens[*count].value[j++] = input[i];
    }

    i++;
}

tokens[*count].value[j] = '\\0';

(*count)++;

if (*count >= MAX_TOKENS)
{
    break;
}
}

void display_tokens(struct Token tokens[], int count)
{
    printf("\\nParser Output:\\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: [%s]\\n",
            i + 1,
            tokens[i].value);
    }
}

int validate_tokens(struct Token tokens[], int count)
{
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut         ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\\ Replace    ^U Paste       ^J Justify    ^/_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 escape_parser.c *
{
    if (count == 0)
    {
        printf("Error: Empty input.\n");
        return 0;
    }

    for (int i = 0; i < count; i++)
    {
        if (strlen(tokens[i].value) == 0)
        {
            printf("Error: Empty token.\n");
            return 0;
        }
    }

    return 1;
}

int main()
{
    char input[MAX_INPUT];

    struct Token tokens[MAX_TOKENS];

    int count;

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

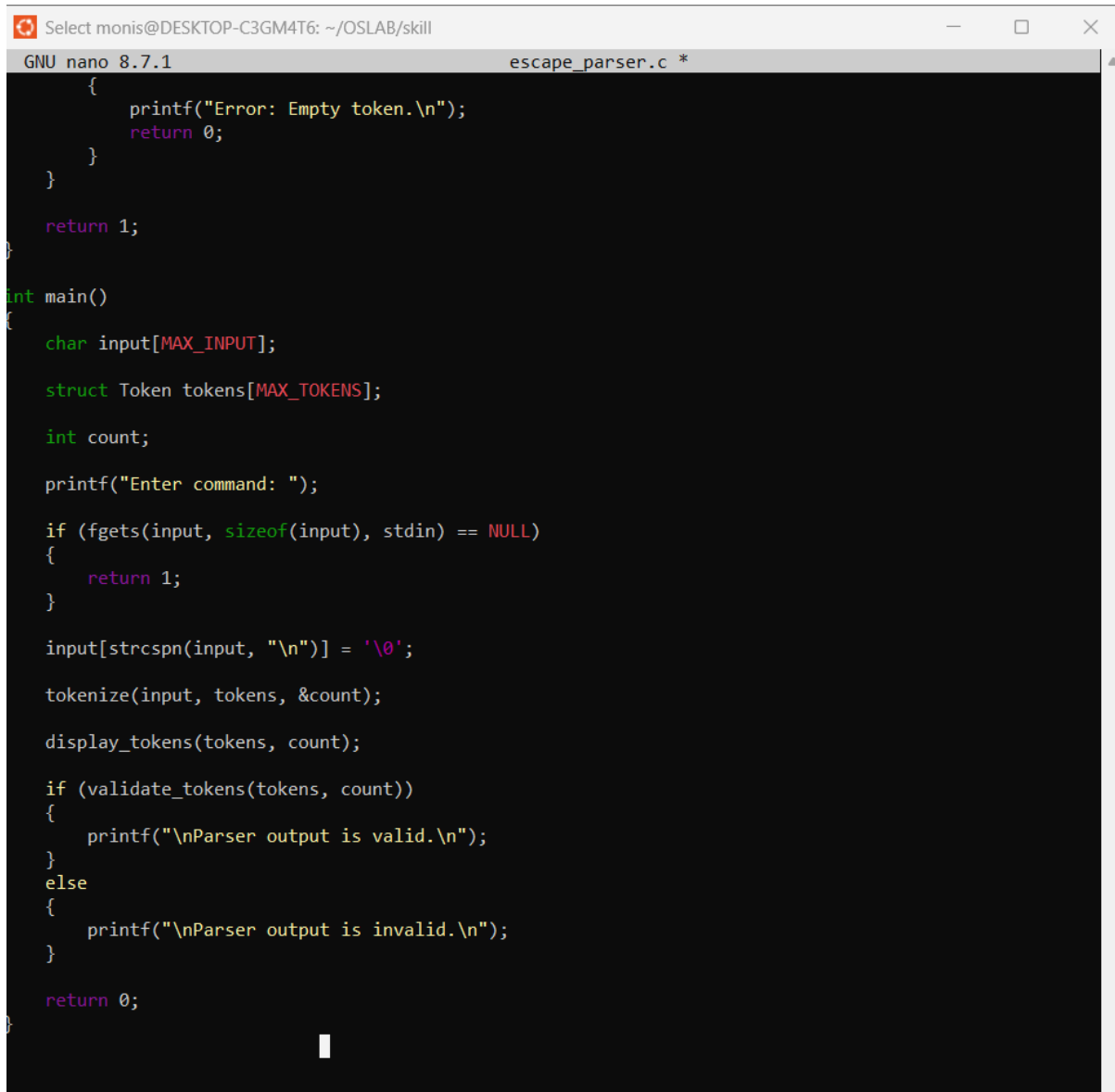
    tokenize(input, tokens, &count);

    display_tokens(tokens, count);

    if (validate_tokens(tokens, count))
    {
        printf("\nParser output is valid.\n");
    }
    else

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-F Redo



```
GNU nano 8.7.1 escape_parser.c *
{
    printf("Error: Empty token.\n");
    return 0;
}
return 1;
}

int main()
{
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKENS];
    int count;

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';
    tokenize(input, tokens, &count);
    display_tokens(tokens, count);

    if (validate_tokens(tokens, count))
    {
        printf("\nParser output is valid.\n");
    }
    else
    {
        printf("\nParser output is invalid.\n");
    }

    return 0;
}
```

Step 3- Save the File

After pasting the code:

Press:

Ctrl + O

Then press:

Enter

Then:

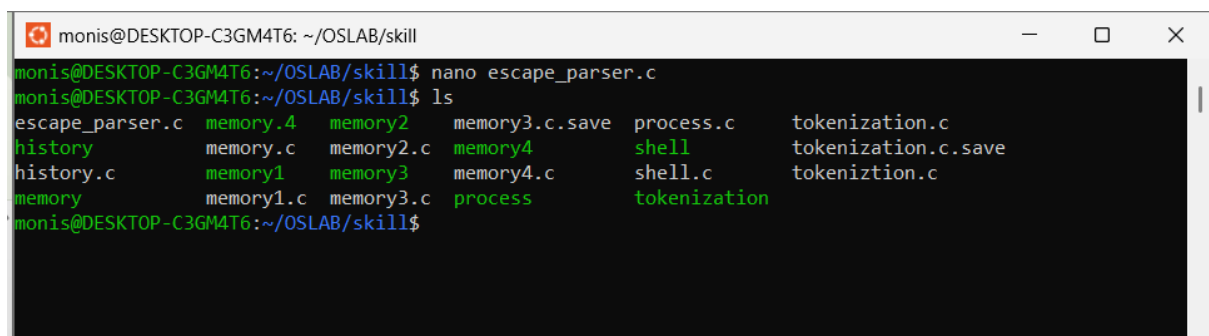
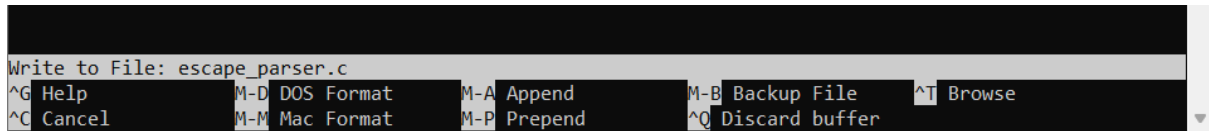
Ctrl + X

Now check:

ls

You should see:

escape_parser.c



Step 4 — Compile

Run:

```
gcc -Wall -g escape_parser.c -o escape_parser
```

If there is **no error message**, compilation succeeded.

Check:

ls -l

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g escape_parser.c -o escape_parser
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l
total 248
-rwxr-xr-x 1 monis monis 20240 Sep 17 19:37 escape_parser
-rw-r--r-- 1 monis monis 2443 Sep 17 19:35 escape_parser.c
-rwxr-xr-x 1 monis monis 16672 Sep 16 09:16 history
-rw-r--r-- 1 monis monis 3804 Sep 16 08:42 history.c
-rwxr-xr-x 1 monis monis 16040 Sep 16 09:28 memory
-rwxr-xr-x 1 monis monis 16128 Sep 16 09:53 memory.4
-rw-r--r-- 1 monis monis 316 Sep 16 09:27 memory.c
-rwxr-xr-x 1 monis monis 16168 Sep 16 09:32 memory1
-rw-r--r-- 1 monis monis 846 Sep 16 09:32 memory1.c
-rwxr-xr-x 1 monis monis 16168 Sep 16 09:36 memory2
-rw-r--r-- 1 monis monis 1002 Sep 16 09:35 memory2.c
-rwxr-xr-x 1 monis monis 16128 Sep 16 09:39 memory3
-rw-r--r-- 1 monis monis 861 Sep 16 09:39 memory3.c
-rw-r--r-- 1 monis monis 861 Sep 16 09:44 memory3.c.save
-rwxr-xr-x 1 monis monis 17736 Sep 16 09:57 memory4
-rw-r--r-- 1 monis monis 1165 Sep 16 09:46 memory4.c
-rwxr-xr-x 1 monis monis 16392 Aug 24 21:35 process
-rw-r--r-- 1 monis monis 894 Aug 24 21:33 process.c
-rwxr-xr-x 1 monis monis 16352 Aug 24 22:20 shell
-rw-r--r-- 1 monis monis 1789 Aug 24 22:19 shell.c
-rwxr-xr-x 1 monis monis 20184 Sep 16 15:26 tokenization
-rw-r--r-- 1 monis monis 2034 Sep 16 15:24 tokenization.c
-rw-r--r-- 1 monis monis 2034 Sep 16 15:19 tokenization.c.save
-rw-r--r-- 1 monis monis 2034 Sep 16 15:20 tokeniztion.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Test normal input

Run:

`./escape_parser`

You will see:

Enter command:

Type:

`ls -l`

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command: ls -l

Parser Output:
Token 1: [ls]
Token 2: [-l]

Parser output is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 6 — Test escaped spaces

Run:

```
./escape_parser
```

Enter:

```
hello\ world
```

Press Enter.

[The \ tells the parser:

Treat the following space as part of the token instead of using it as a separator.

This demonstrates:

Handle Escaped Spaces]

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command: hello\ world

Parser Output:
Token 1: [hello world]

Parser output is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 7 — Test escaped special symbols

Run:

```
./escape_parser
```

Enter:

hello\|world

Press Enter.

The backslash protects the | character.

So:

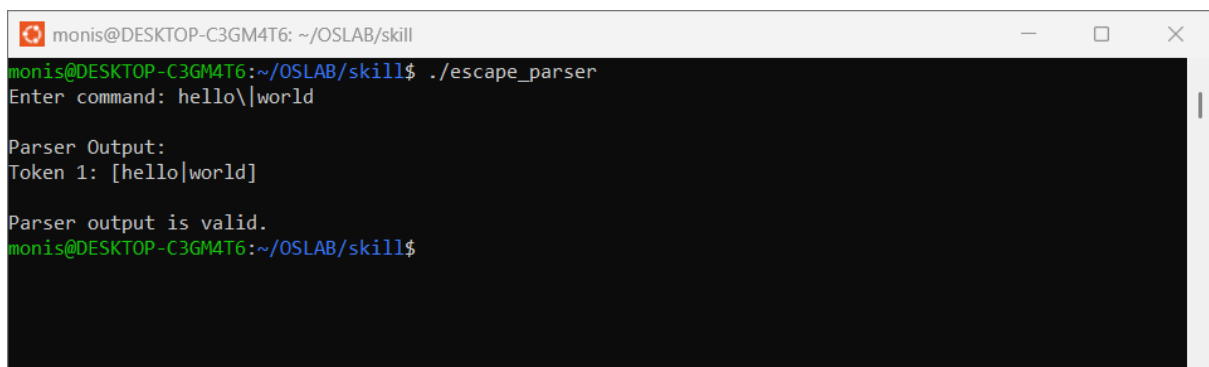
hello\|world

becomes:

hello|world

This demonstrates:

Escape Special Symbols

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The prompt is 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'. The user enters './escape_parser'. The program outputs 'Enter command: hello\|world'. Then it outputs 'Parser Output:' followed by 'Token 1: [hello|world]'. Finally, it outputs 'Parser output is valid.' and returns to the prompt 'monis@DESKTOP-C3GM4T6:~/OSLAB/skill\$'.

Step 8 — Test preservation of characters

Run:

./escape_parser

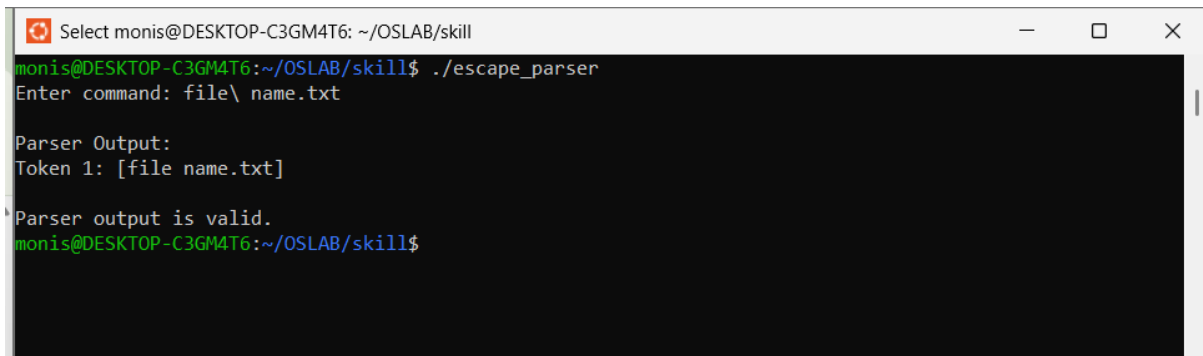
Enter:

file\ name.txt

[The space was preserved inside the token.

This demonstrates:

Preserve Characters]



```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command: file\ name.txt

Parser Output:
Token 1: [file name.txt]

Parser output is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 9 — Test complex input

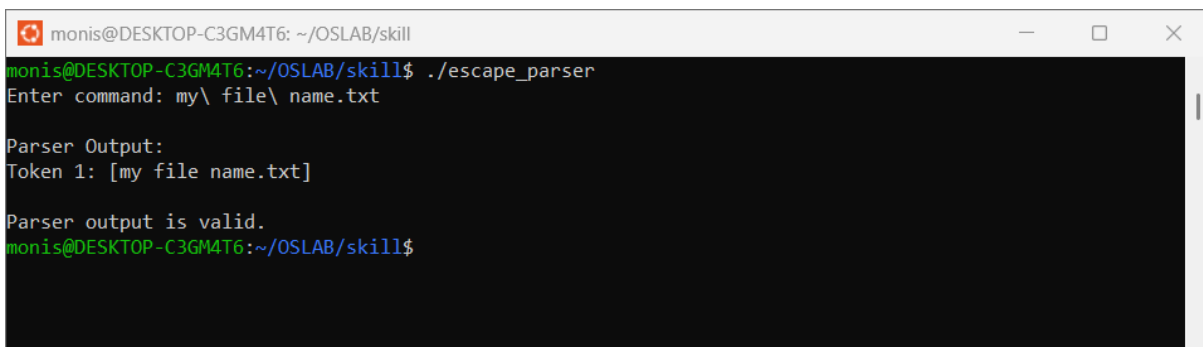
Run:

```
./escape_parser
```

Enter:

```
my\ file\ name.txt
```

[All the escaped spaces remain inside one token.]



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command: my\ file\ name.txt

Parser Output:
Token 1: [my file name.txt]

Parser output is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 10 — Test multiple tokens with an escaped space

Run:

```
./escape_parser
```

Enter:

```
cp my\ file.txt /tmp
```

[This is a very good test because it demonstrates both:

- normal token separation
- escaped space inside a token]

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command: cp my\ file.txt /tmp

Parser Output:
Token 1: [cp]
Token 2: [my file.txt]
Token 3: [/tmp]

Parser output is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 11 — Test empty input

Run:

`./escape_parser`

When it says:

Enter command:

just press Enter.

[This demonstrates parser validation.]

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command:

Parser Output:
Error: Empty input.

Parser output is invalid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 12 — Test the complex input

Run:

`./escape_parser`

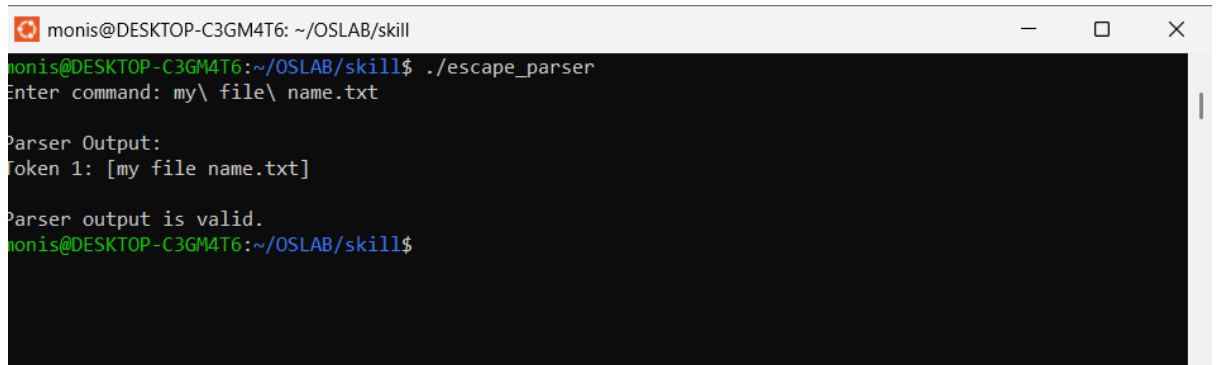
When it asks:

Enter command:

type exactly:

my\ file\ name.txt

Then press Enter.

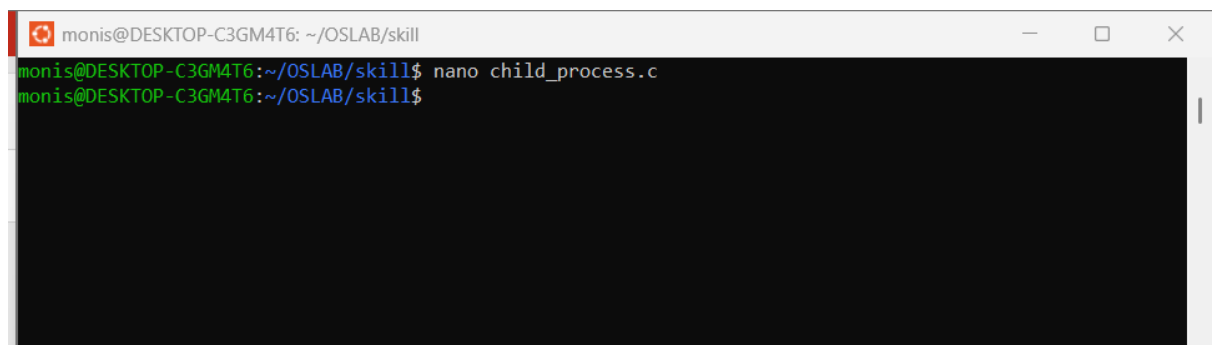


```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./escape_parser
Enter command: my\ file\ name.txt

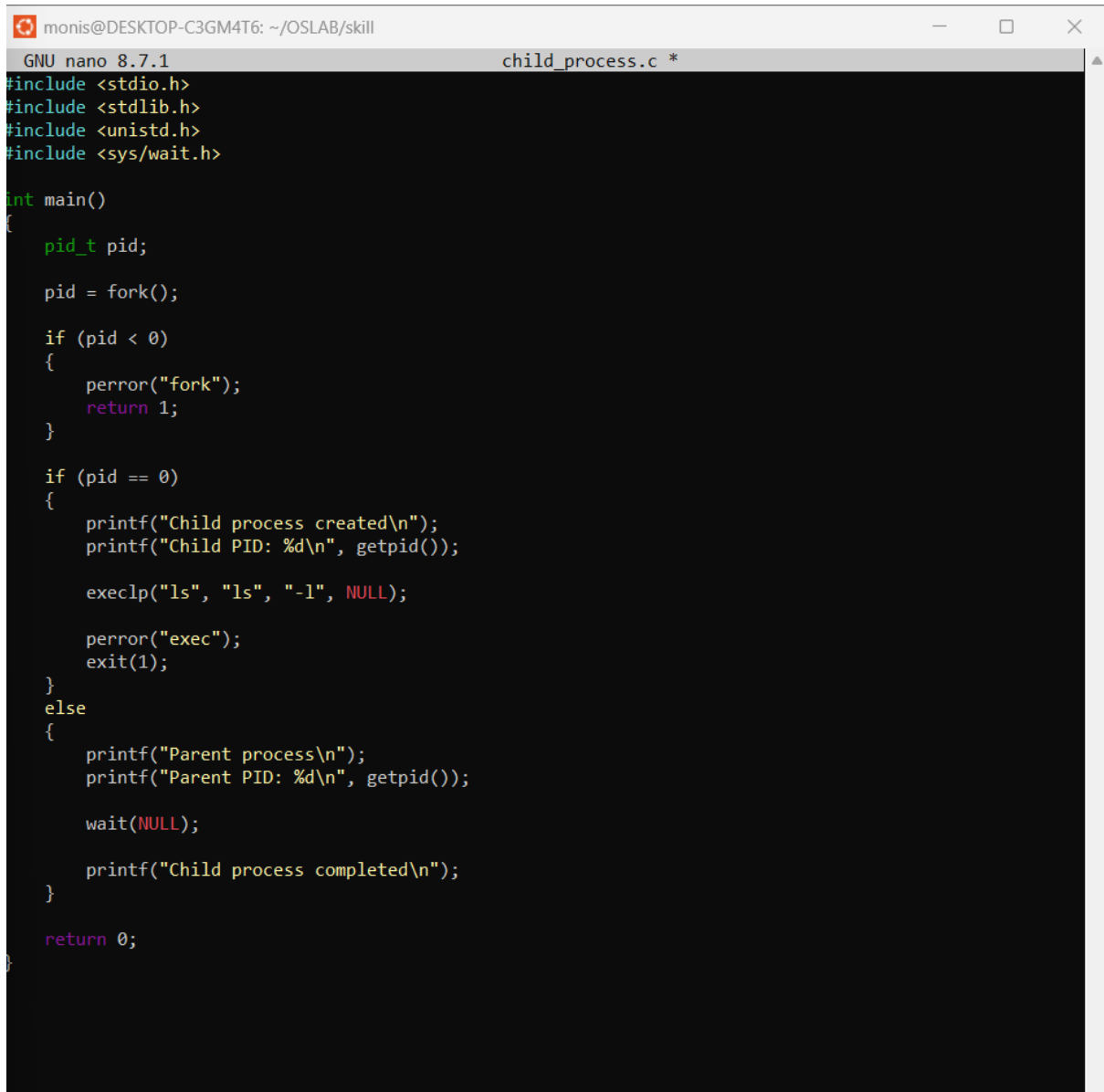
Parser Output:
Token 1: [my file name.txt]

Parser output is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 13 — Create the Child Process Program



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano child_process.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 child_process.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process created\n");
        printf("Child PID: %d\n", getpid());

        execlp("ls", "ls", "-l", NULL);

        perror("exec");
        exit(1);
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

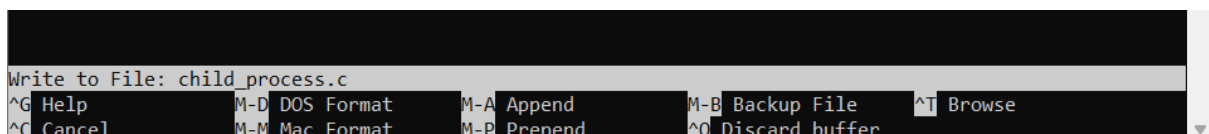
        printf("Child process completed\n");
    }

    return 0;
}
```

Save the file

In nano:

1. Press **Ctrl + O**
2. Press **Enter**
3. Press **Ctrl + X**



Step 14- Compile the Child Process Program

In the same terminal, run:

```
gcc -Wall -g child_process.c -o child_process
```

If there are no errors, you will simply return to the prompt, like:

```
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Then check that the executable was created:

```
ls -l child_process.c child_process
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano child_process.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g child_process.c -o child_process
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

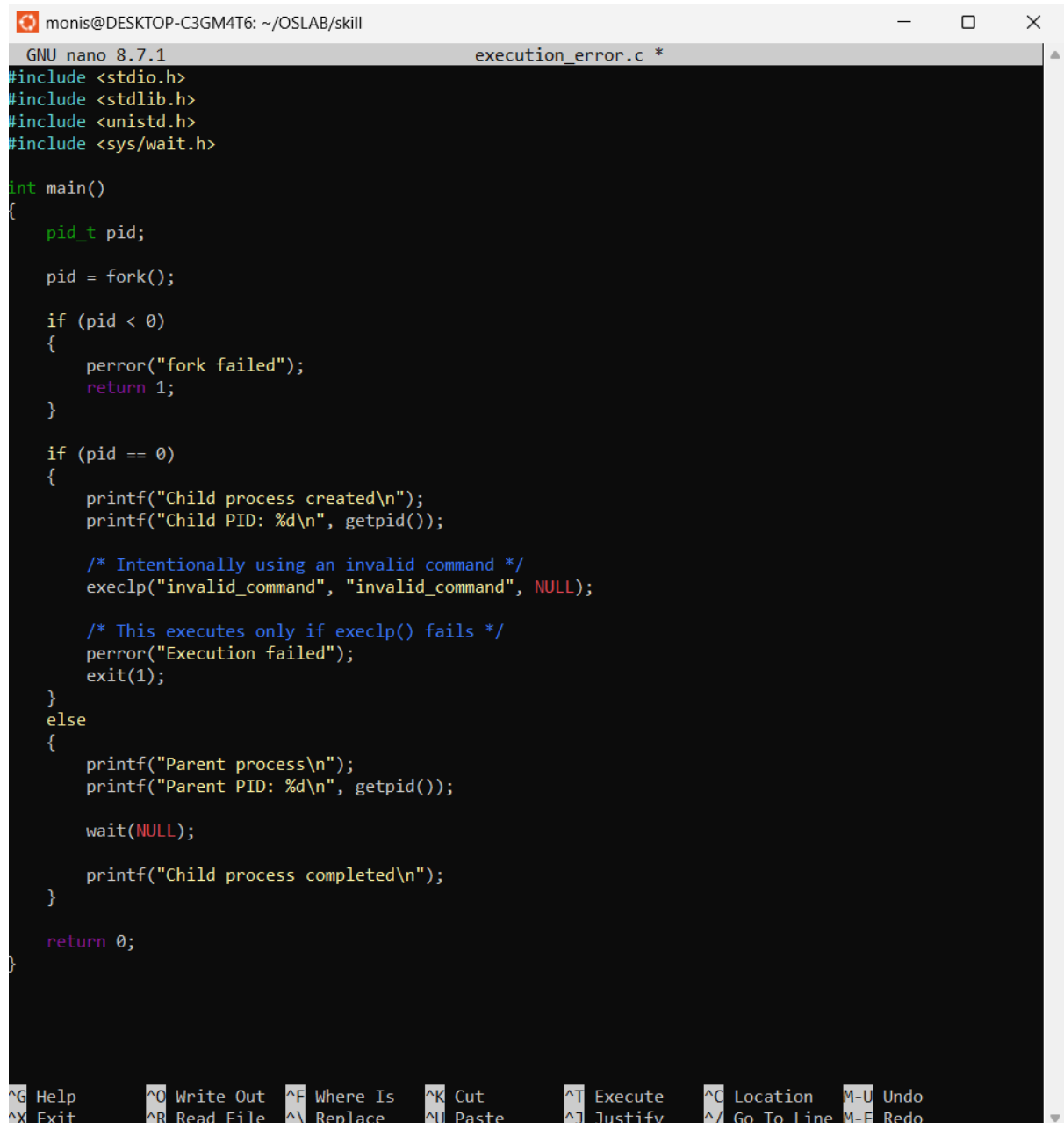
```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano child_process.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g child_process.c -o child_process
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l child_process.c child_process
-rwxr-xr-x 1 monis monis 18000 Sep 17 19:56 child_process
-rw-r--r-- 1 monis monis  590 Sep 17 19:56 child_process.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 16 — Test Execution Error Handling

Create a new file:

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano execution_error.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Paste this complete code:



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 execution_error.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process created\n");
        printf("Child PID: %d\n", getpid());

        /* Intentionally using an invalid command */
        execvp("invalid_command", "invalid_command", NULL);

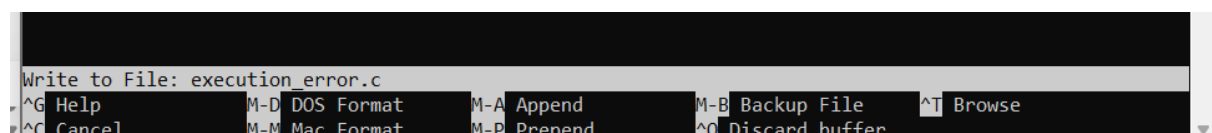
        /* This executes only if execvp() fails */
        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child process completed\n");
    }

    return 0;
}
```

Save
Press:
Ctrl + O
Enter
Ctrl + X



```
Write to File: execution_error.c
^G Help      ^M-D DOS Format  ^M-A Append     ^M-B Backup File ^T Browse
^C Cancel    ^M-M Mac Format  ^M-P Prepend    ^O Discard buffer
```

Compile:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano execution_error.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g execution_error.c -o execution_error
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Run:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano execution_error.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g execution_error.c -o execution_error
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./execution_error
Parent process
Parent PID: 4024
Child process created
Child PID: 4025
Execution failed: No such file or directory
Child process completed
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 17 — Pass Arguments to the Child Process

Now create a separate program, so your previous programs stay unchanged.

Run:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pass_arguments.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Save the file

Press:

Ctrl + O

Enter

Ctrl + X

Step 18 — Compile `pass_arguments.c`

Run:

`gcc -Wall -g pass_arguments.c -o pass_arguments`

If there are no errors, check that the executable was created:

ls -l pass_arguments.c pass_arguments
[you should see both files listed.]

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pass_arguments.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g pass_arguments.c -o pass_arguments
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l pass_arguments.c pass_arguments
-rwxr-xr-x 1 monis monis 18000 Sep 17 20:07 pass_arguments
-rw-r--r-- 1 monis monis 677 Sep 17 20:05 pass_arguments.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 19 – Run the Argument-Passing Program

Now run:

./pass_arguments

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pass_arguments.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g pass_arguments.c -o pass_arguments
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l pass_arguments.c pass_arguments
-rwxr-xr-x 1 monis monis 18000 Sep 17 20:07 pass_arguments
-rw-r--r-- 1 monis monis 677 Sep 17 20:05 pass_arguments.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pass_arguments
Parent process
Parent PID: 4123
Child process created
Child PID: 4124
Hello from child process
Child process completed
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 20- Manage the Parent Process

Create a separate program:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano parent_child.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parent_child.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());

        sleep(2);

        printf("Child process finished\n");
        exit(0);
    }
    else
    {
        printf("Parent process started\n");
        printf("Parent PID: %d\n", getpid());

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }

        printf("Parent process finished\n");
    }

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parent_child.c *
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());

        sleep(2);

        printf("Child process finished\n");
        exit(0);
    }
    else
    {
        printf("Parent process started\n");
        printf("Parent PID: %d\n", getpid());

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }

        printf("Parent process finished\n");
    }

    return 0;
}
```

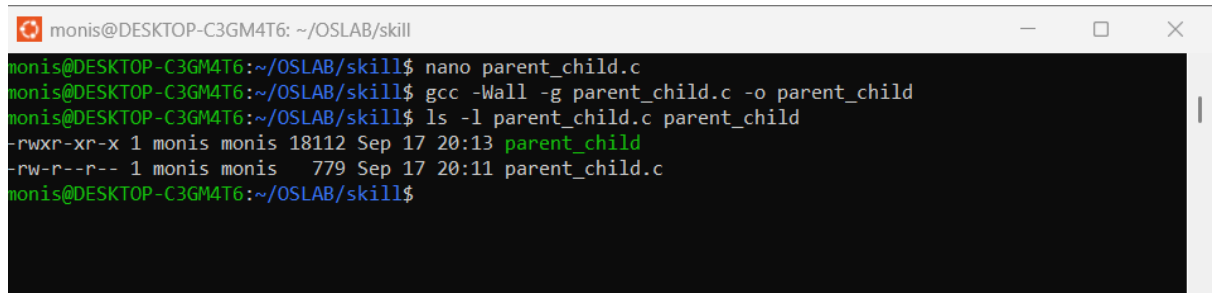
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save
Press:
Ctrl + O
Enter
Ctrl + X

```
Write to File: parent_child.c
^G Help   M-D DOS Format   M-A Append   M-B Backup File   ^T Browse
^C Cancel   M-M Mac Format   M-P Prepend   ^Q Discard buffer
```

Step 21 — Compile parent_child.c
Run:
gcc -Wall -g parent_child.c -o parent_child
If there are no errors, check the files:

ls -l parent_child.c parent_child

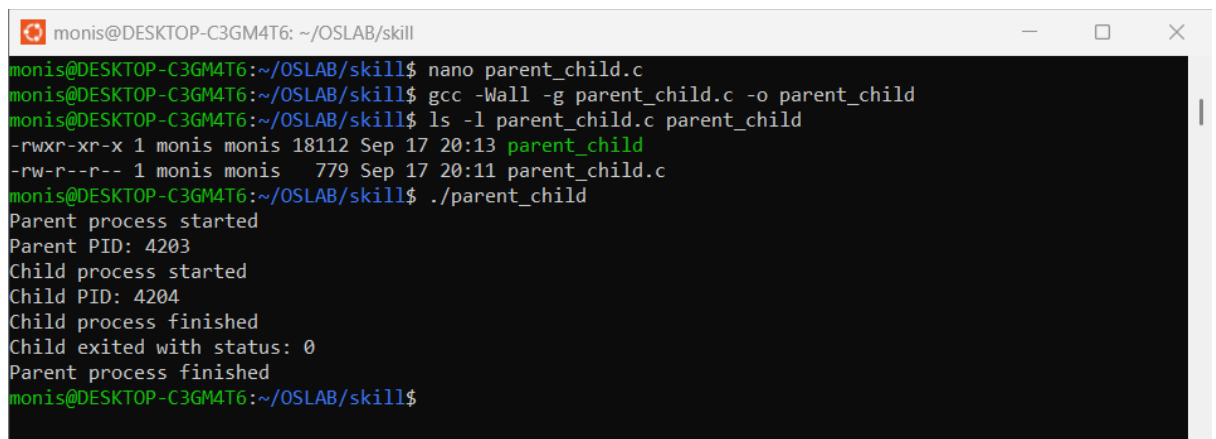


```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano parent_child.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g parent_child.c -o parent_child
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l parent_child.c parent_child
-rwxr-xr-x 1 monis monis 18112 Sep 17 20:13 parent_child
-rw-r--r-- 1 monis monis 779 Sep 17 20:11 parent_child.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 22 — Run the Parent–Child Program

Now run:

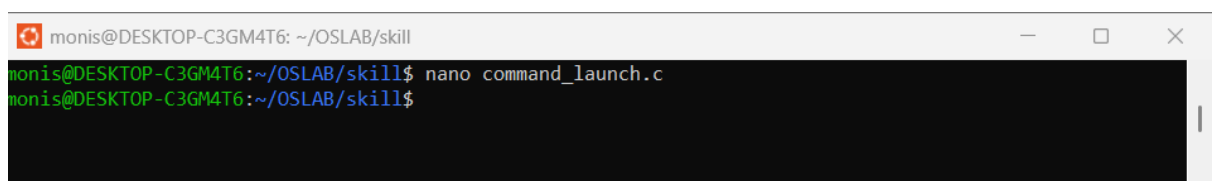
`./parent_child`



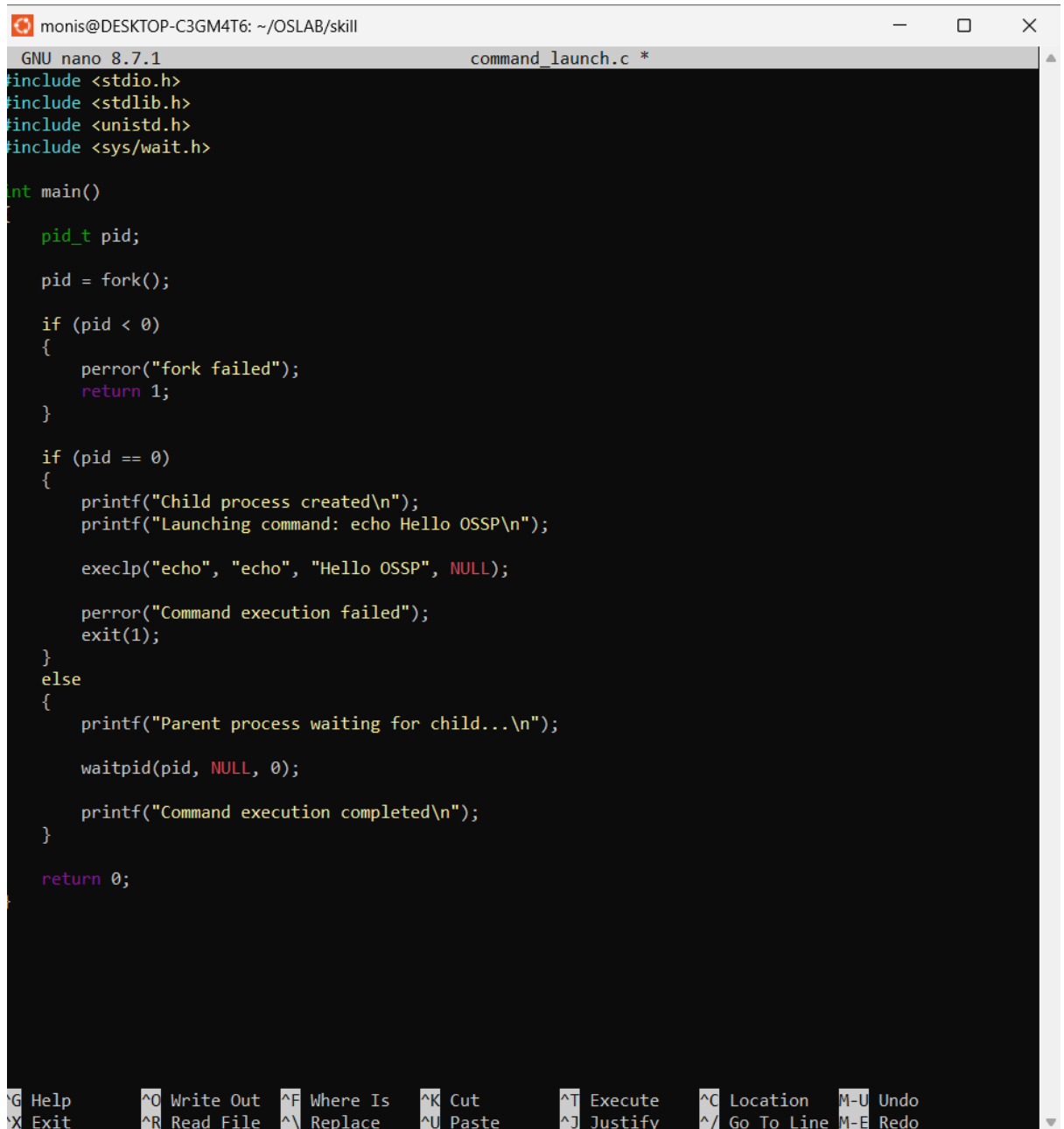
```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano parent_child.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g parent_child.c -o parent_child
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l parent_child.c parent_child
-rwxr-xr-x 1 monis monis 18112 Sep 17 20:13 parent_child
-rw-r--r-- 1 monis monis 779 Sep 17 20:11 parent_child.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parent_child
Parent process started
Parent PID: 4203
Child process started
Child PID: 4204
Child process finished
Child exited with status: 0
Parent process finished
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 23 — Test Command Launching

Create:



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano command_launch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 command_launch.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process created\n");
        printf("Launching command: echo Hello OSSP\n");

        execlp("echo", "echo", "Hello OSSP", NULL);

        perror("Command execution failed");
        exit(1);
    }
    else
    {
        printf("Parent process waiting for child...\n");

        waitpid(pid, NULL, 0);

        printf("Command execution completed\n");
    }

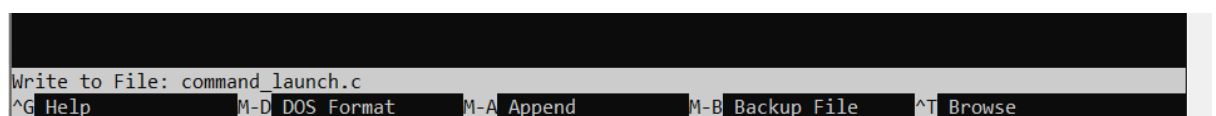
    return 0;
}
```

Save with:

Ctrl + O

Enter

Ctrl + X



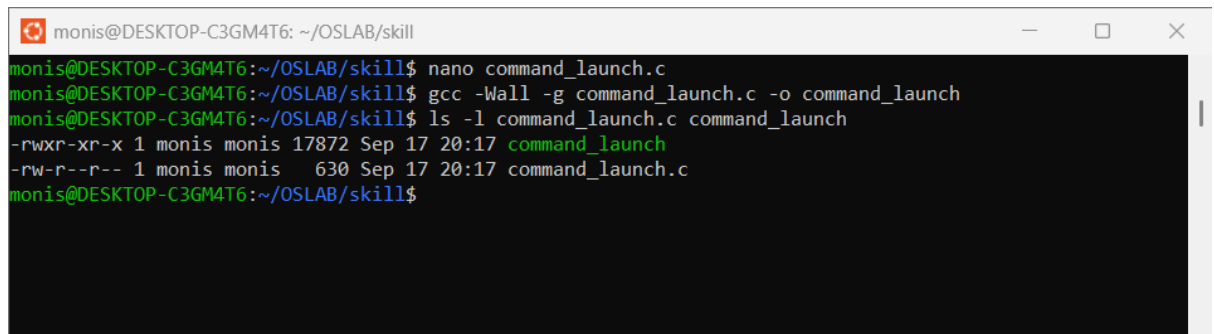
```
Write to File: command_launch.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
```

Step 24 — Compile `command_launch.c`

Run:

```
gcc -Wall -g command_launch.c -o command_launch
```

If there are no errors, check that the executable was created:
`ls -l command_launch.c command_launch`

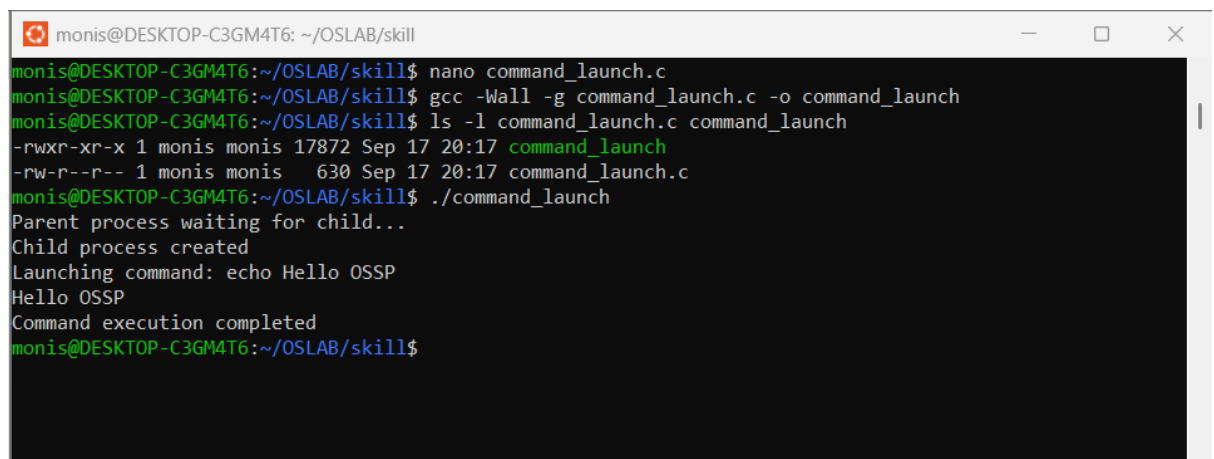


```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano command_launch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g command_launch.c -o command_launch
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l command_launch.c command_launch
-rwxr-xr-x 1 monis monis 17872 Sep 17 20:17 command_launch
-rw-r--r-- 1 monis monis  630 Sep 17 20:17 command_launch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 25 — Run the Command-Launching Program

Now run:

`./command_launch`



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano command_launch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g command_launch.c -o command_launch
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l command_launch.c command_launch
-rwxr-xr-x 1 monis monis 17872 Sep 17 20:17 command_launch
-rw-r--r-- 1 monis monis  630 Sep 17 20:17 command_launch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./command_launch
Parent process waiting for child...
Child process created
Launching command: echo Hello OSSP
Hello OSSP
Command execution completed
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```